

■ 2.8.2 Memory

Time is usually not a critical limitation in the computations that you do. At worst, you can always just wait longer to get your results. Memory space, on the other hand, can represent an insuperable barrier in a calculation.

If the intermediate expressions that you would generate in a particular calculation are larger than the memory you have available, then you will simply never be able to do that calculation on the computer system you are using.

Mathematica is quite careful about the way that it uses memory. Every time an intermediate expression that you generate is no longer needed, *Mathematica* immediately frees the memory that was allocated for it. (In technical terms, *Mathematica* uses a version of reference count memory management; as a result, it does not, for example, have to do “garbage collection”.)

<code>MemoryInUse[]</code>	the number of bytes of memory space currently being used by <i>Mathematica</i>
<code>MaxMemoryUsed[]</code>	the maximum number of bytes of memory used in this <i>Mathematica</i> session
<code>ByteCount[expr]</code>	the number of bytes of memory used to store <i>expr</i>
<code>LeafCount[expr]</code>	the number of terminal nodes in the expression tree for <i>expr</i>
<code>Share[expr]</code>	optimize the storage of <i>expr</i>
<code>Share[]</code>	optimize the storage of all expressions

Memory management functions.

One issue that often comes up is exactly how much memory *Mathematica* can actually use on a particular computer system. Usually there is a certain amount of memory available for *all* processes running on the computer at a particular time. Sometimes this amount of memory is equal to the physical number of bytes of RAM in the computer. Often, it includes a certain amount of “virtual memory”, obtained by swapping data on and off a mass storage device.

When *Mathematica* runs, it needs space both for data and for code. The complete code of *Mathematica* is typically at least one megabyte in size. For any particular calculation, only a small fraction of this code is usually used. However, in trying to work out the total amount of space available for *Mathematica* data, you should not forget what is needed for *Mathematica* code. In addition, you must

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

include the space that is taken up by other processes running in the computer. If there are fewer jobs running, you will usually find that your job can use more memory.

It is also worth realizing that the time needed to do a calculation can depend very greatly on how much physical memory you have. Although virtual memory allows you in principle to use large amounts of memory space, it is usually hundreds or even thousands of times slower to access than physical memory. As a result, if your calculation becomes so large that it needs to make use of virtual memory, it may run *much* more slowly.

Another point to realize is that under most operating systems, *Mathematica* can never really free memory that has once been allocated to it. The result from `MaximumMemoryUsed[ ]` therefore usually gives not only the maximum amount of memory that your current *Mathematica* session has used, but also the total amount that the operating system has allocated to it.

There are various ways that you can save memory in a *Mathematica* session. The first is to remove values that you are no longer using. As soon you remove all references to a particular expression, *Mathematica* will free the memory associated with that expression.

You should remember, however, that any expression you got as output is stored as the value of the corresponding `Out[n]`. You can use `Clear[Out]` to get rid of these references to expressions.

The second way to save memory in storing expressions is to use the function `Share`. To see what `Share` does, you must understand some things about how *Mathematica* actually stores expressions.

Consider an expression like `f[1 + x, 1 + x]`. The two arguments `1 + x` are identical expressions. The question is whether internally, they are both actually stored in the same piece of computer memory, or whether there are two copies of `1 + x` at different places in computer memory. What `Share` does is to make sure that all expressions that are identical are actually stored in the same piece of computer memory. It effectively “shares” the memory that is used to store expressions.

The results that you get from `ByteCount` give the number of bytes that a particular expression takes *assuming that none of its subexpressions are shared*. As a result, `ByteCount` often gives you a substantial overestimate of the actual number of bytes of computer memory that a particular expression really uses.